

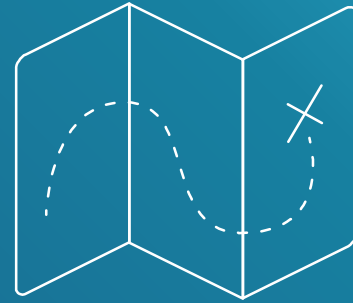
Deep Learning Pipelines for Apache Spark



Belsabel Woldemichael

Contents

- ◇ Introduction
- ◇ Design
- ◇ Implementation
- ◇ Test
- ◇ Enhancement Ideas
- ◇ Conclusion





Introduction

Project Overview: Introduce Deep Learning Pipelines for Apache Spark, a new library developed by Databricks. This library provides high-level APIs for scalable deep learning model application and transfer learning by integrating popular deep learning libraries with MLlib Pipelines and Spark SQL.

Goals:

- Ensure scalable application of deep learning models.
- Enable transfer learning through integration with MLlib Pipelines and Spark SQL.
- Facilitate scalable image processing and analysis using deep learning.

Key Technologies Used:

- **Apache Spark:** For distributed data processing and scalable machine learning model training.
- **MLlib Pipelines:** Builds and tunes complex machine learning workflows.
- **Spark SQL:** Enables SQL querying of structured data, integrates with DLP for applying deep learning models.
- **Deep Learning Libraries (TensorFlow, Keras):** Defines and trains models, facilitates transfer learning and feature extraction in Spark.



Design

- **Image Data Handling:**

- Tools for native image handling in Spark DataFrames.
- Supports reading, processing, and transforming image data at scale.

- **Transfer Learning:**

- Adapt pre-trained deep learning models to new tasks.
- Utilize existing models for feature extraction and fine-tuning.

- **Model Application at Scale:**

- APIs for applying deep learning models to large datasets for predictions or feature transformations.
- Integration with Spark's distributed processing ensures efficient execution.

- **Deploying Models as SQL Functions:**

- Upcoming feature to use deep learning models directly within SQL queries.
- Enables non-programmers to leverage deep learning models through SQL.

- **Distributed Hyper-Parameter Tuning:**

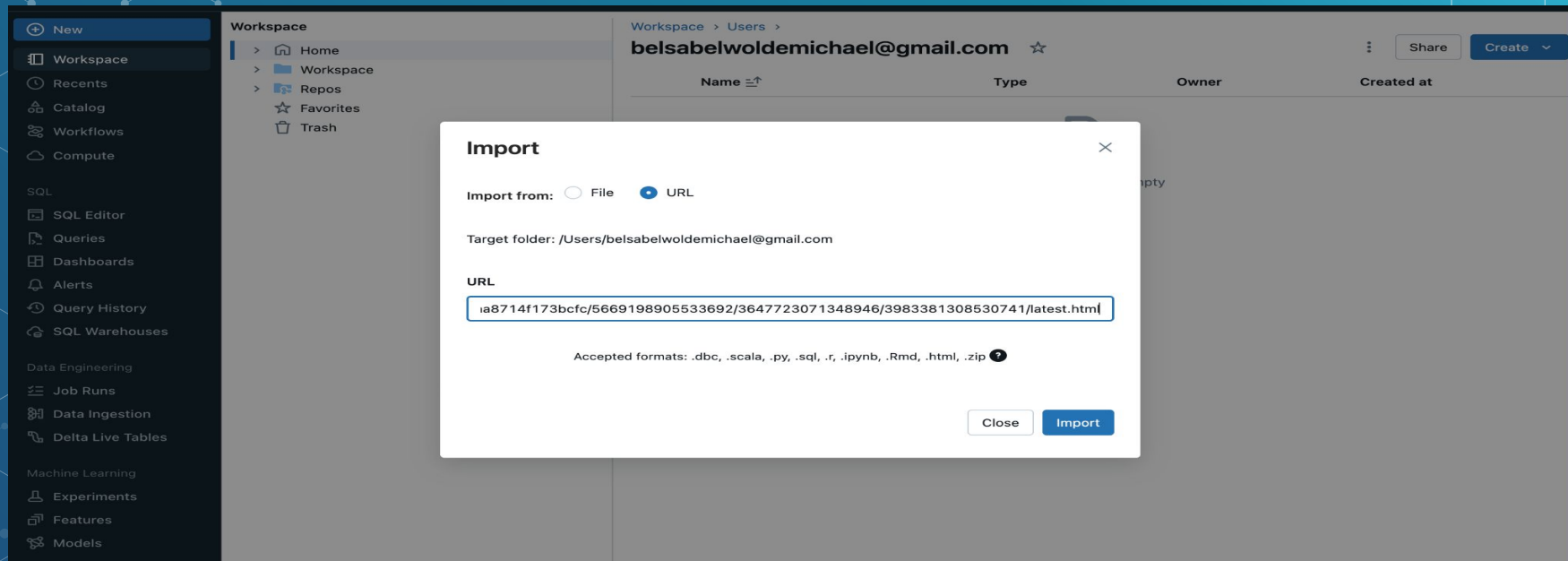
- Planned feature for optimizing model parameters using Spark MLlib Pipelines.
- Automates tuning to achieve optimal model performance.



Implementation and Test

1. Create an account in databricks and create a workspace and import the notebook

<https://databricks-prod-cloudfront.cloud.databricks.com/public/4027ec902e239c93eaaa8714f173bcfc/5669198905533692/3647723071348946/3983381308530741/latest.html>



Step 1: Setting Up the Cluster Environment

- **Install Libraries:**

- Deep Learning Pipelines is available as a Spark Package. To use it on your cluster:
 - Create a new library with the source option "Maven Coordinate".
 - Use "Search Spark Packages and Maven Central" to find "spark-deep-learning".
 - Attach the library to your cluster.

- **Install Dependencies:**

- Add the Maven coordinate for spark-deep-learning to your Spark setup.
- Install required Python libraries: TensorFlow, Keras, h5py, and spark-deep-learning.

Step 2: Obtain Image Dataset

- Download and Extract Dataset: We will use lowers dataset from the TensorFlow

```
1/22/2018 (4s) 5 Python

%sh
curl -O http://download.tensorflow.org/example_images/flower_photos.tgz
tar xzf flower_photos.tgz

% Total      % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total   Spent    Left   Speed

  0     0    0     0    0     0      0  0 --:--:-- --:--:-- --:--:--    0
  0     0    0     0    0     0      0  0 --:--:-- --:--:-- --:--:--    0
12 218M  12 28.0M    0    24.0M    0 0:00:09 0:00:01 0:00:08 24.0M
100 218M 100 218M    0    114M    0 0:00:01 0:00:01 --:--:-- 114M

tar: flower_photos/roses/14810868100_87eb739f26_m.jpg: Cannot change ownership to uid 270850, gid 5000: Invalid argument
tar: flower_photos/roses/1446090416_f0cad5fde4.jpg: Cannot change ownership to uid 270850, gid 5000: Invalid argument
tar: flower_photos/roses/15319767030_e6c5602a77_m.jpg: Cannot change ownership to uid 270850, gid 5000: Invalid argument
tar: flower_photos/roses/15032112248_30c5284e54_n.jpg: Cannot change ownership to uid 270850, gid 5000: Invalid argument
tar: flower_photos/roses/7211616670_2d49ecb3a5_m.jpg: Cannot change ownership to uid 270850, gid 5000: Invalid argument
tar: flower_photos/roses/15674450867_0ced942941_n.jpg: Cannot change ownership to uid 270850, gid 5000: Invalid argument
tar: flower_photos/roses/17158274118_00ec99a23c.jpg: Cannot change ownership to uid 270850, gid 5000: Invalid argument
tar: flower_photos/roses/14019883858_e5d2a0ec10_n.jpg: Cannot change ownership to uid 270850, gid 5000: Invalid argument
tar: flower_photos/roses/8035908422_87220425d2_n.jpg: Cannot change ownership to uid 270850, gid 5000: Invalid argument
tar: flower_photos/roses/14747962886_2bff6bb323_m.jpg: Cannot change ownership to uid 270850, gid 5000: Invalid argument
tar: flower_photos/roses/4356781875_92c5cd93c0.jpg: Cannot change ownership to uid 270850, gid 5000: Invalid argument
tar: flower_photos/roses/8524595546_b242bd4928_n.jpg: Cannot change ownership to uid 270850, gid 5000: Invalid argument
tar: flower_photos/roses/9486573080_60eab9278e_n.jpg: Cannot change ownership to uid 270850, gid 5000: Invalid argument
tar: flower_photos/roses/6039330368_c30ed224c4_m.jpg: Cannot change ownership to uid 270850, gid 5000: Invalid argument
```

```
1/22/2018 (4s) 6 Python

display(dbutils.fs.ls('file:/Workspace/Users/belsabelwoldemichael@gmail.com/flower_photos'))
```

	path	name	size
1	file:/databricks/driver/flower_photos/daisy/	daisy/	32768
2	file:/databricks/driver/flower_photos/dandelion/	dandelion/	49152
3	file:/databricks/driver/flower_photos/sunflowers/	sunflowers/	36864
4	file:/databricks/driver/flower_photos/LICENSE.txt	LICENSE.txt	418049
5	file:/databricks/driver/flower_photos/tulips/	tulips/	40960
6	file:/databricks/driver/flower_photos/roses/	roses/	36864

6 rows | 3.64 seconds runtime

Refreshed 2,385 days ago

Step 3: Create a Sample Set of Images

```
# Let's create a small sample set of images for quick demonstrations.
sample_img_dir = img_dir + "/sample"
dbutils.fs.mkdirs(sample_img_dir)
files = dbutils.fs.ls(img_dir + "/tulips")[0:1] + dbutils.fs.ls(img_dir + "/daisy")[0:2]
for f in files:
    dbutils.fs.cp(f.path, sample_img_dir)
display(dbutils.fs.ls(sample_img_dir))
```

Table ▾ +

🔍 ⚙ 📄

	^A _C path	^A _C name	¹ ₂ ³ size
1	dbfs:/tmp/flower_photos/sample/100080576_f52e8ee070_n.jpg	100080576_f52e8ee070_n.jpg	26797
2	dbfs:/tmp/flower_photos/sample/100930342_92e8746431_n.jpg	100930342_92e8746431_n.jpg	26200
3	dbfs:/tmp/flower_photos/sample/10140303196_b88d3d6cec.jpg	10140303196_b88d3d6cec.jpg	117247

⬇ 3 rows | 5.49 seconds runtime

Refreshed 2,385 days ago

Step 4: Load Images into Spark DataFrames • Read Images: Use DLP's tools to read images directly into Spark DataFrames.

- The resulting DataFrame contains a string column named "filePath" containing the path to each image file, and a image struct ("SpImage") column called "image" containing the decoded image data

1/22/2018 (1s)

10

```
from sparkdl import readImages
image_df = readImages(sample_img_dir)
```

image_df: pyspark.sql.dataframe.DataFrame = [filePath: string, image: struct]

```
/databricks/python/local/lib/python2.7/site-packages/h5py/__init__.py:36: FutureWarning: Conversion of the second argument of
issubdtype from `float` to `np.floating` is deprecated. In future, it will be treated as `np.float64 == np.dtype(float).type`
`
from ._conv import register_converters as _register_converters
Using TensorFlow backend.
```

The resulting DataFrame contains a string column named "filePath" containing the path to each image file, and a image struct (" SpImage ") column called "image" containing the decoded image data.

1/22/2018 (4s)

12

Python

```
display(image_df)
```

Table

	filePath	image
1	dbfs:/tmp/flower_photos/sample/100080576_f52e8ee070_n.jpg	> {"mode":"RGB","height":263,"width":320,"nChannels":3,"data":"h4eFioqljo6OkZGRkpKS56
2	dbfs:/tmp/flower_photos/sample/100930342_92e8746431_n.jpg	> {"mode":"RGB","height":209,"width":320,"nChannels":3,"data":"Ey4PEC8QDi8QDi8SES4SE

Step 5: Create Training and Test Data Frames for Transfer Learning

- Data Preparation: ○ Read images, assign labels, and split the data into training and test sets

✓ 1/22/2018 (1s)

14

```
# Create training & test DataFrames for transfer learning - this piece of code is longer than transfer learning itself
below!
from sparkdl import readImages
from pyspark.sql.functions import lit

tulips_df = readImages(img_dir + "/tulips").withColumn("label", lit(1))
daisy_df = readImages(img_dir + "/daisy").withColumn("label", lit(0))
tulips_train, tulips_test, _ = tulips_df.randomSplit([0.05, 0.05, 0.9]) # use larger training sets (e.g. [0.6, 0.4] for
non-community edition clusters)
daisy_train, daisy_test, _ = daisy_df.randomSplit([0.05, 0.05, 0.9]) # use larger training sets (e.g. [0.6, 0.4] for
non-community edition clusters)
train_df = tulips_train.unionAll(daisy_train)
test_df = tulips_test.unionAll(daisy_test)

# Under the hood, each of the partitions is fully loaded in memory, which may be expensive.
# This ensure that each of the paritions has a small size.
train_df = train_df.repartition(100)
test_df = test_df.repartition(100)
```

Step 6: Train and Evaluate the Model

- Model Training and Evaluation: ○ Train the model using a pipeline that includes feature extraction with a pre-trained model and logistic regression, then evaluate its performance

```
1/22/2018 (10m) 15 Python

from pyspark.ml.classification import LogisticRegression
from pyspark.ml import Pipeline
from sparkdl import DeepImageFeaturizer

featurizer = DeepImageFeaturizer(inputCol="image", outputCol="features", modelName="InceptionV3")
lr = LogisticRegression(maxIter=20, regParam=0.05, elasticNetParam=0.3, labelCol="label")
p = Pipeline(stages=[featurizer, lr])

p_model = p.fit(train_df)

Downloading data from https://github.com/fchollet/deep-learning-models/releases/download/v0.5/inception_v3_weights_tf_dim_
ering_tf_kernels_notop.h5

16384/87910968 [.....] - ETA: 0s
24576/87910968 [.....] - ETA: 7:06
57344/87910968 [.....] - ETA: 5:49
73728/87910968 [.....] - ETA: 6:31
106496/87910968 [.....] - ETA: 5:48
139264/87910968 [.....] - ETA: 5:27
196608/87910968 [.....] - ETA: 4:36
262144/87910968 [.....] - ETA: 4:01
352256/87910968 [.....] - ETA: 3:26
458752/87910968 [.....] - ETA: 2:59
557056/87910968 [.....] - ETA: 2:45
696320/87910968 [.....] - ETA: 2:26
860160/87910968 [.....] - ETA: 2:10
1032192/87910968 [.....] - ETA: 1:58
1220608/87910968 [.....] - ETA: 1:49
1433600/87910968 [.....] - ETA: 1:40
1671168/87910968 [.....] - ETA: 1:31
1833312/87910968 [.....] - ETA: 1:22
```

```
1/22/2018 (11m) 18 Python

from pyspark.ml.evaluation import MulticlassClassificationEvaluator

tested_df = p_model.transform(test_df)
evaluator = MulticlassClassificationEvaluator(metricName="accuracy")
print("Test set accuracy = " + str(evaluator.evaluate(tested_df.select("prediction", "label"))))

> tested_df: pyspark.sql.dataframe.DataFrame

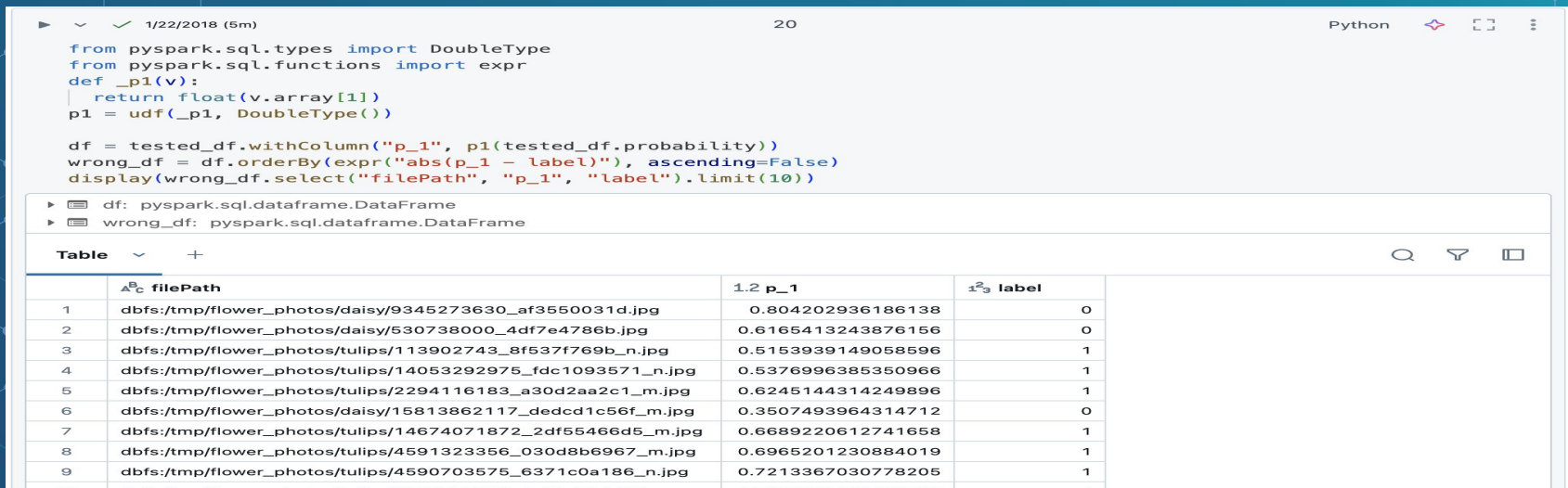
INFO:tensorflow:Froze 376 variables.
Converted 376 variables to const ops.
INFO:tensorflow:Froze 0 variables.
Converted 0 variables to const ops.
Test set accuracy = 0.971014492754
```

Step 7: Applying Deep Learning Models at Scale

- **Spark DataFrames:** Use for large-scale dataset model applications.
- **Transformers:** Apply TensorFlow Graphs and Keras Models at scale with Deep Learning Pipelines.
- **Pre-Trained Models:** Apply popular image models without TensorFlow or Keras code using Tensorframes library.

Step 7.1: Apply Popular Image Models

- Use **DeepImagePredictor** to apply pre-trained models for image classification.



The screenshot shows a Jupyter Notebook interface with a code cell and a table view of a Spark DataFrame. The code cell contains the following Python code:

```
from pyspark.sql.types import DoubleType
from pyspark.sql.functions import expr
def _p1(v):
    return float(v.array[1])
p1 = udf(_p1, DoubleType())

df = tested_df.withColumn("p_1", p1(tested_df.probability))
wrong_df = df.orderBy(expr("abs(p_1 - label)"), ascending=False)
display(wrong_df.select("filePath", "p_1", "label").limit(10))
```

Below the code cell, the variable `wrong_df` is shown as a `pyspark.sql.dataframe.DataFrame`. The table view displays the following data:

	filePath	p_1	label
1	dbfs:/tmp/flower_photos/daisy/9345273630_af3550031d.jpg	0.804202936186138	0
2	dbfs:/tmp/flower_photos/daisy/530738000_4df7e4786b.jpg	0.6165413243876156	0
3	dbfs:/tmp/flower_photos/tulips/113902743_8f537f769b_n.jpg	0.5153939149058596	1
4	dbfs:/tmp/flower_photos/tulips/14053292975_fdc1093571_n.jpg	0.5376996385350966	1
5	dbfs:/tmp/flower_photos/tulips/2294116183_a30d2aa2c1_m.jpg	0.6245144314249896	1
6	dbfs:/tmp/flower_photos/daisy/15813862117_dedcd1c56f_m.jpg	0.3507493964314712	0
7	dbfs:/tmp/flower_photos/tulips/14674071872_2df55466d5_m.jpg	0.6689220612741658	1
8	dbfs:/tmp/flower_photos/tulips/4591323356_030d8b6967_m.jpg	0.6965201230884019	1
9	dbfs:/tmp/flower_photos/tulips/4590703575_6371c0a186_n.jpg	0.7213367030778205	1

- The `predicted\_labels` column shows that "daisy" is a high-probability class for all the sample flowers using the base model. However, the neural network can distinguish between the flower types based on the differences in the probability values. This demonstrates that our transfer learning example was able to learn and recognize the differences between daisies and tulips effectively, starting from the base model.

▶ ▼ ✓ 1/22/2018 (38s)

25

Python ✨ [] ⋮

```
df = p_model.transform(image_df)
display(df.select("filePath", (1-p1(df.probability)).alias("p_daisy")))
```

▶ 📄 df: pyspark.sql.dataframe.DataFrame

Table ▼ +

🔍 🔍 🗪

	^A _C filePath	1.2 p_daisy
1	dbfs:/tmp/flower_photos/sample/100080576_f52e8ee070_n.jpg	0.968875532556084
2	dbfs:/tmp/flower_photos/sample/100930342_92e8746431_n.jpg	0.13976502414604564
3	dbfs:/tmp/flower_photos/sample/10140303196_b88d3d6cec.jpg	0.9786112803439984

⌵ 3 rows | 38.01 seconds runtime

Refreshed 2,385 days ago

Step 7.2: Create Custom TensorFlow Graphs - (For TensorFlow users)

- The TFImageTransformer allows users to apply custom TensorFlow Graphs to the image DataFrame. This is useful for specialized tasks where pre-trained models may not suffice.

1/22/2018 (2s)

27

Python



```
from sparkdl import readImages, TFImageTransformer
from sparkdl.transformers import utils
import tensorflow as tf

image_df = readImages(sample_img_dir)

g = tf.Graph()
with g.as_default():
    image_arr = utils.imageInputPlaceholder()
    resized_images = tf.image.resize_images(image_arr, (299, 299))
    # the following step is not necessary for this graph, but can be for graphs with variables, etc
    frozen_graph = utils.stripAndFreezeGraph(g.as_graph_def(add_shapes=True), tf.Session(graph=g), [resized_images])

transformer = TFImageTransformer(inputCol="image", outputCol="transformed_img", graph=frozen_graph,
                                inputTensor=image_arr, outputTensor=resized_images,
                                outputMode="image")
tf_trans_df = transformer.transform(image_df)
```

image_df: pyspark.sql.dataframe.DataFrame = [filePath: string, image: struct]

tf_trans_df: pyspark.sql.dataframe.DataFrame = [filePath: string, image: struct ... 1 more field]

INFO:tensorflow:Froze 0 variables.
Converted 0 variables to const ops.
INFO:tensorflow:Froze 0 variables.
Converted 0 variables to const ops.

Step 7.3: Apply Keras Models (For Keras users)

- The KerasImageFileTransformer allows users to apply Keras models to the DataFrame containing image URIs. This is useful for leveraging custom-trained Keras models within the Spark Ecosystem.

▶ ✓ 1/22/2018 (2m) 30

```
from keras.applications import InceptionV3

model = InceptionV3(weights="imagenet")
model.save('/tmp/model-full.h5') # saves to the local filesystem
# move to a permanent place for future use
dbfs_model_path = 'dbfs:/models/model-full.h5'
dbutils.fs.cp('file:/tmp/model-full.h5', dbfs_model_path)
```

Out[14]: True

Step 8: Testing with Pre-trained Keras Model:

- This code snippet illustrates how to apply a pre-trained Keras model (InceptionV3) to images stored in a DataFrame.
- Images are loaded and preprocessed according to the input requirements of the InceptionV3 model.
- The pre-trained model is applied to the preprocessed images to generate predictions.
- The use of Spark DataFrames and transformers ensures that the process can scale to handle large datasets efficiently

```
1/22/2018 (22s) 32 Python
```

```
from keras.applications.inception_v3 import preprocess_input
from keras.preprocessing.image import img_to_array, load_img
import numpy as np
from pyspark.sql.types import StringType
from sparkdl import KerasImageFileTransformer

def loadAndPreprocessKerasInceptionV3(uri):
    # this is a typical way to load and prep images in keras
    image = img_to_array(load_img(uri, target_size=(299, 299))) # image dimensions for InceptionV3
    image = np.expand_dims(image, axis=0)
    return preprocess_input(image)

dbutils.fs.cp(dbfs_model_path, 'file:/tmp/model-full-tmp.h5')
transformer = KerasImageFileTransformer(inputCol="uri", outputCol="predictions",
                                       modelFile="/tmp/model-full-tmp.h5", # local file path for model
                                       imageLoader=loadAndPreprocessKerasInceptionV3,
                                       outputMode="vector")

files = ["/dbfs" + str(f.path)[5:] for f in dbutils.fs.ls(sample_img_dir)] # make "local" file paths for images
uri_df = sqlContext.createDataFrame(files, StringType()).toDF("uri")

keras_pred_df = transformer.transform(uri_df)
```

```
uri_df: pyspark.sql.dataframe.DataFrame = [uri: string]
keras_pred_df: pyspark.sql.dataframe.DataFrame

/databricks/python/local/lib/python2.7/site-packages/keras/models.py:255: UserWarning: No training configuration found in save file: the model was *not* compiled. Compile it manually.
warnings.warn('No training configuration found in save file: '
INFO:tensorflow:Froze 378 variables.
Converted 378 variables to const ops.
INFO:tensorflow:Froze 0 variables.
Converted 0 variables to const ops.
```

This table displays the results of applying the pre-trained Keras model to the images in the sample directory using the KerasImageFileTransformer. Here's an explanation of what each column and the data within them represent:

- uri: This column lists the file paths (URIs) of the images that were processed.
- predictions: This column contains the model's predictions for each image. Each prediction includes a list of probabilities corresponding to different classes.

▶ ▾ ✓ 1/22/2018 (8s) 33 Python ✨ ⌵ ⋮

display(keras_pred_df.select("uri", "predictions"))

Table ▾ + 🔍 ⌵ □

	uri	predictions
1	/dbfs/tmp/flower_photos/sample/100080576_f52e8ee070_n.jpg	> [1,1000,[],[0.00007469155389117077,0.00007630845357198268,0.000193578365724
2	/dbfs/tmp/flower_photos/sample/100930342_92e8746431_n.jpg	> [1,1000,[],[0.0002563267189543694,0.0028356011025607586,0.0001203265346703
3	/dbfs/tmp/flower_photos/sample/10140303196_b88d3d6cec.jpg	> [1,1000,[],[0.00003744077548617497,0.000053084160754224285,0.00009704380495

⌵

⬇ 3 rows | 7.68 seconds runtime Refreshed 2,385 days ago



Enhancement Ideas

Deploying Models as SQL Functions:

- Allow deep learning models to be accessed and used via SQL queries.

Distributed Hyper-Parameter Tuning:

- Optimize model performance through distributed hyper-parameter tuning using Spark MLlib Pipelines.

Support for Additional Deep Learning Libraries:

- Extend compatibility to include frameworks like PyTorch and MXNet.

Enhanced Image Processing Capabilities:

- Improve functionality for processing and augmenting image data.



Conclusion

- Successfully integrated Deep Learning Pipelines with Apache Spark for scalable deep learning solutions.
- Efficiently loaded, processed, and transformed large-scale image datasets using Spark DataFrames.
- Implemented transfer learning with pre-trained models like InceptionV3 for feature extraction and fine-tuning.
- Trained and evaluated logistic regression models using Spark MLlib Pipelines, achieving high accuracy on test datasets.
- Demonstrated the benefits of Spark for distributed deep learning workflows in terms of scalability and performance.
- Created a streamlined workflow for deep learning model application, covering data ingestion to model evaluation.

References

Introducing Deep Learning Pipelines for Apache Spark

Github link

https://github.com/BelsabelTekle/Cloud_Computing/tree/main/Kubernetes/Machine_learning/Apache+Deep_learning